Komplain.ai

REFINED QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

Team Name: AS

Team ID: 178

UMHackathon 2026 Final Round

Domain 1: AI Systems & Agentic Workflow Automation

Date: 3rd of May 2026

© UMHackathon 2026 Email: umhackathon@um.edu.my

Document Control

Field	Detail
System Under Test (SUT)	Komplain.ai — AI-Powered Complaint Resolution Copilot (UMHackathon 2026)
GitHub Repository	https://github.com/Ph0enix19/Komplain.AI
Live Frontend (Vercel)	https://komplain-test-xi2r.vercel.app
Live Backend (Render)	https://komplaintest.onrender.com/api/health
CI/CD Pipeline	GitHub Actions - lint (ruff) / test (pytest 55 passing) / security (pip-audit + detect-secrets) / https://github.com/Ph0enix19/Komplain.AI/actions
Postman Collection (QATD)	postman_qatd_collection.json - 10 requests / 22 pm.test assertions (committed to repository root)
Postman Collection (Core)	postman_collection.json - 4 requests / 9 pm.test assertions (committed to repository root)
Document Version	v3.0 — Final Round Refined QATD (Accuracy-Corrected)
Submission Date	3 May 2026

QA Objective

The primary objective of this Refined QATD is to demonstrate that Komplain.ai — a 5-agent AI-powered customer complaint resolution copilot — has been tested with sufficient rigour across all critical pipelines. GLM (Z.ai) acts as the central reasoning engine. If the GLM component is removed, the system loses its ability to coordinate and execute the complaint workflow. The QA process covered:

- Manual execution of happy-path and edge-case scenarios, including Manglish input handling and AI output anti-hallucination validation.
- Automated pytest suite: 55 passing tests, 0 failures, across four core modules (API, LLM client, agents, storage). Command: python -m pytest -q. Result: 55 passed in 0.77s.
- Integration verification via postman_qatd_collection.json (10 requests / 22 pm.test assertions) covering all /api/* endpoints including telemetry fields, /events, and /stream.
- CI/CD enforcement via GitHub Actions on every push to main (lint + test + security — 3 jobs).
- Systematic defect tracking with root cause analysis and commit-level traceability for 6 confirmed defects.
- 5 known technical debt items transparently documented with GitHub issue links.

© UMHackathon 2026
Email: umhackathon@um.edu.my

Judging Criteria Alignment

This QATD is mapped to the UMHackathon 2026 judging rubric under Technical Architecture & Deployment — Quality Assurance & Reliability Execution (15% total):

Judging Sub-Category	Weight	Evidence in This Document
Test Execution & Automation Evidence	5%	5 manual test cases (TC-01 to TC-AI-01), 55 pytest tests, 10 Postman QATD requests / 22 assertions, GitHub Actions CI
Defect Traceability & Risk Follow-Through	5%	DEF-01 to DEF-06 with root cause analysis, reproduction steps, fix descriptions, verified commit references
Transparency & Architectural Trade-off	5%	TD-01 to TD-05: JSON storage cap, no auth, no rate limiting, provider risk, no frontend tests — all with GitHub issue links

© UMHackathon 2026
Email: umhackathon@um.edu.my

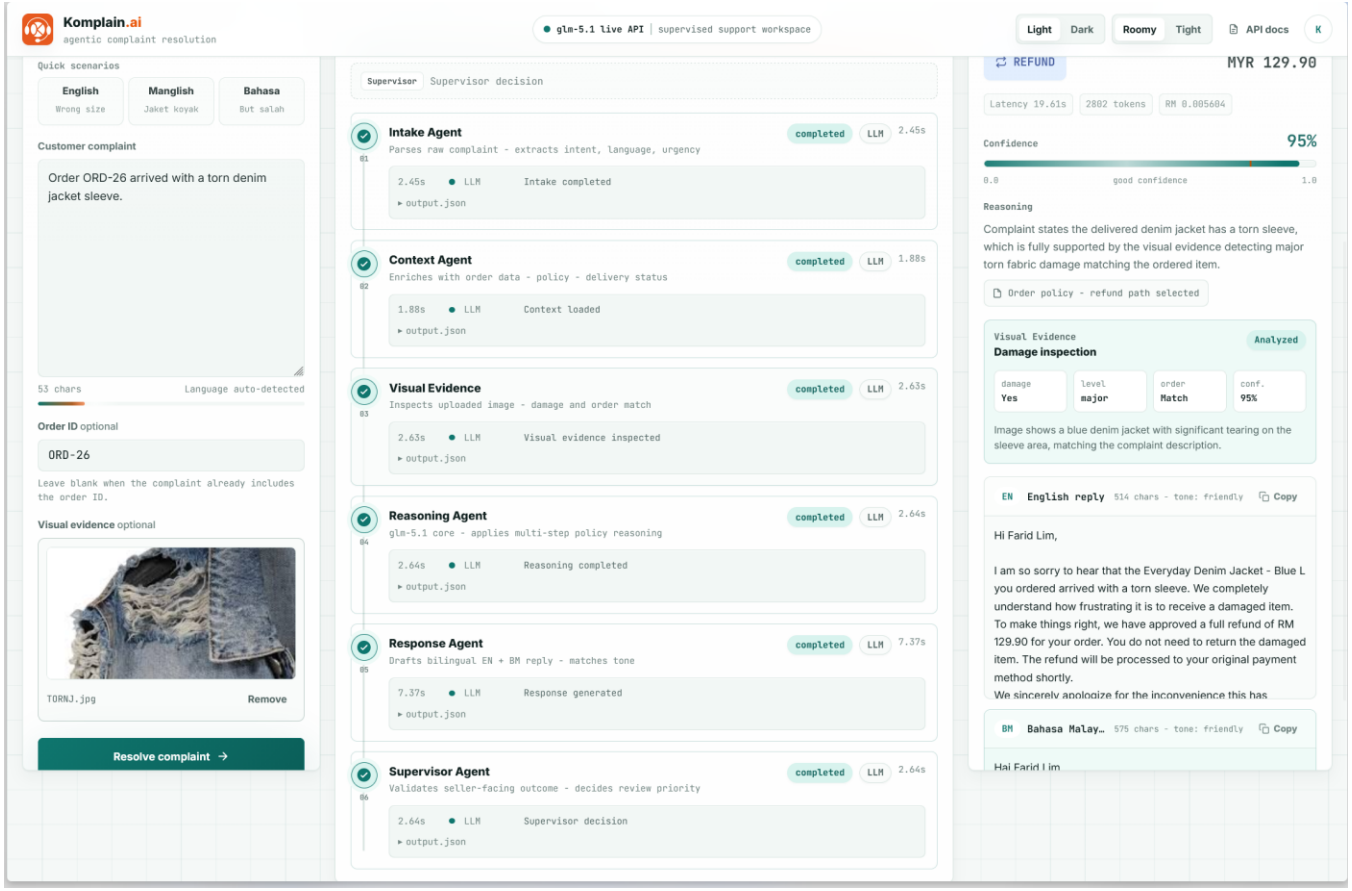
1. Test Execution & Evidence (Manual)

The following test cases were executed manually on the deployed system (Vercel frontend + Render backend). Tests cover happy paths, negative cases, and AI output validation.

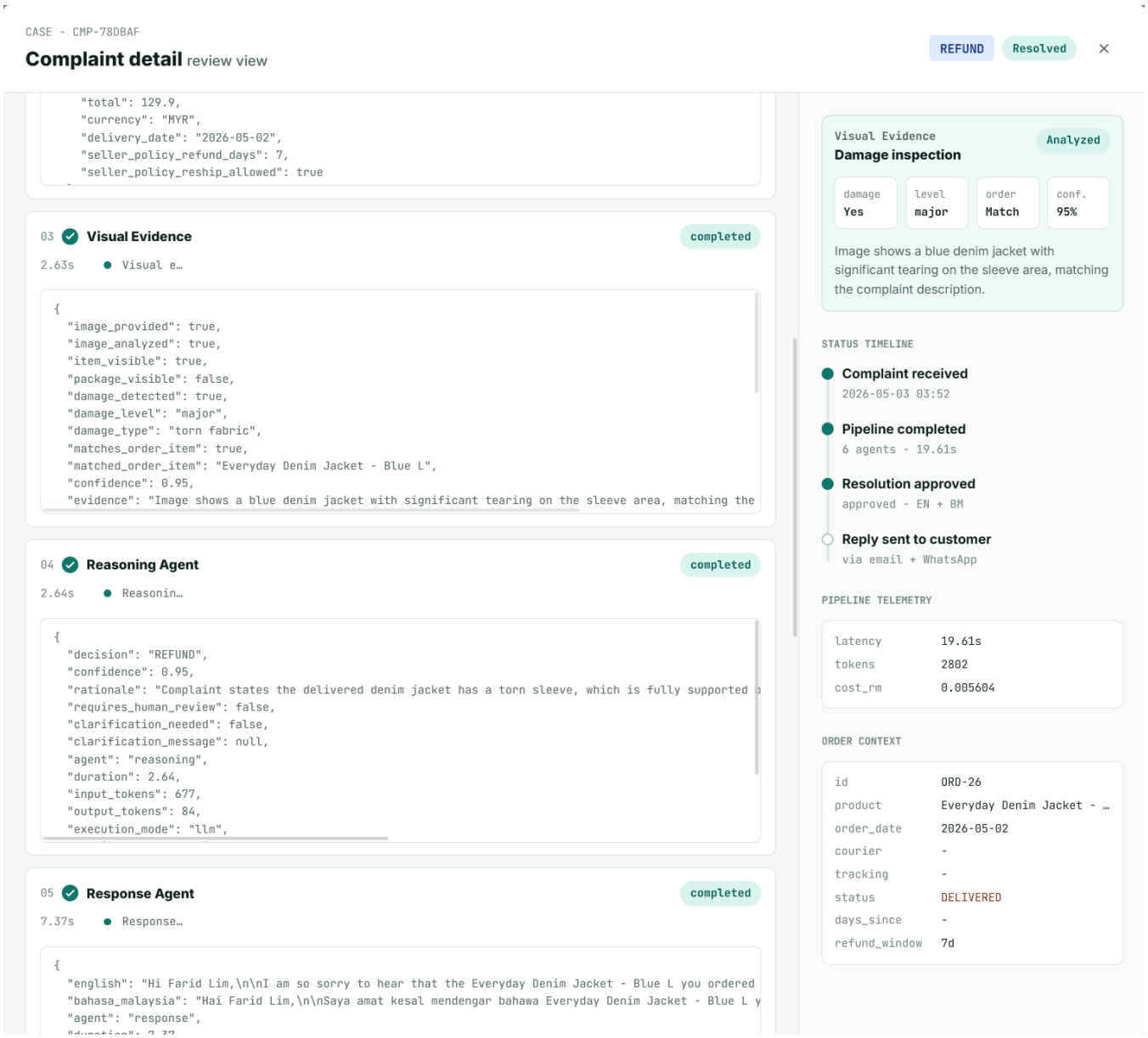
TC-01 — Happy Path: Full Pipeline (English Complaint) (With Vision Model)

Field	Detail
Type	Happy Path — Full Pipeline (English)
Input	Complaint: 'Order ORD-26 arrived with a torn denim jacket sleeve.' | Order ID: ORD-26 (seeded in data/orders.json)
Expected Result	POST /api/complaints returns HTTP 200, status=PROCESSING immediately. Background pipeline runs all 6 agents. Completed record includes: intake.language=EN, reasoning.decision=REFUND or RESHIP, response.english and response.bahasa_malaysia both non-empty, supervisor.requires_human_review set, total_latency and estimated_cost_rm present.
Actual Result	POST returned HTTP 200, status=PROCESSING, id assigned. Pipeline completed. Decision: REFUND. Bilingual reply generated correctly in English and Bahasa Malaysia. Latest live telemetry range: total_latency 15.77s-33.88s, total_tokens 2,133-2,896, estimated_cost_rm RM 0.004266-RM 0.005792.
Status	**PASS**

© UMHackathon 2026
Email: umhackathon@um.edu.my



© UMHackathon 2026 Email: umhackathon@um.edu.my



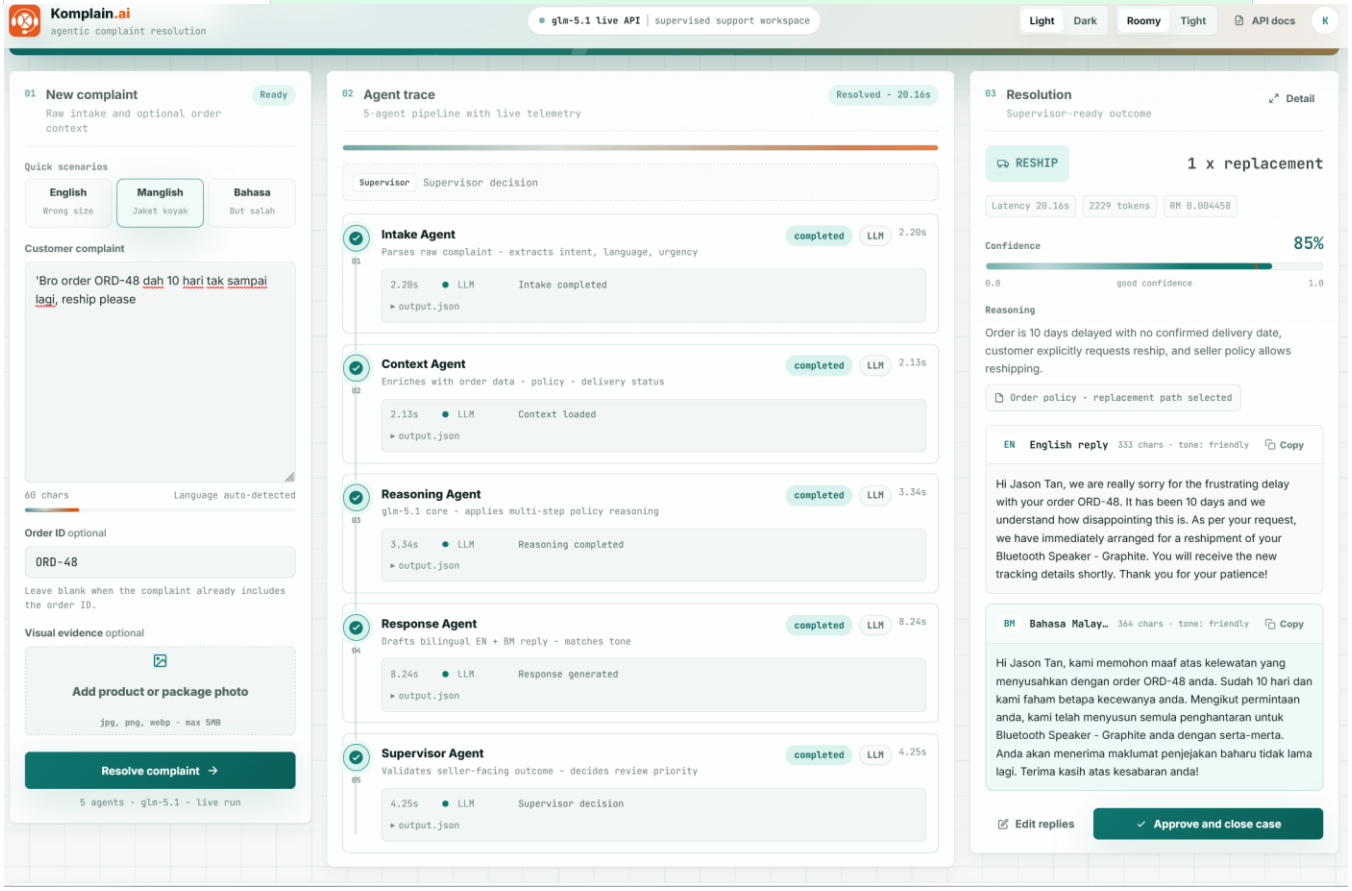
Screenshots showing the testing done on the deployed website

TC-02 — Happy Path: Full Pipeline (Manglish Complaint) (Without Vision Model)

Field	Detail
Type	Happy Path — Full Pipeline (Manglish)
Input	Complaint: 'Bro order ORD-48 dah 10 hari tak sampai lagi, reship please' | Order ID: ORD-48 (seeded in data/orders.json)

© UMHackathon 2026 Email: umhackathon@um.edu.my

Field	Detail
Expected Result	Intake agent detects intake.language=Manglish and intake.order_id=ORD-48. Context agent retrieves order data for ORD-48. Response agent produces BM-primary bilingual reply (response.bahasa_malaysia and response.english). Supervisor sets supervisor.requires_human_review and supervisor.priority.
Actual Result	Language correctly identified as Manglish. Order ORD-48 found. Decision: RESHIP. BM-primary bilingual reply generated. All 5 agents completed successfully.
Status	**PASS**



© UMHackathon 2026 Email: umhackathon@um.edu.my

CASE - CMP-F88D61

Complaint detail review view

RESHIP Resolved ×


```
    "seller_policy_reship_allowed": true
  },
  "notes": "Order found for Jason Tan. Status is IN_TRANSIT with no confirmed delivery date.",
  "agent": "context",
  "duration": 2.13,
  "input_tokens": 256,
  "output_tokens": 36,
  "execution_mode": "llm",
  "provider_used": "zai",
  "fallback_used": false
}
```


03 **Reasoning Agent** completed

3.34s • Reasonin…


```
  "rationale": "Order is 10 days delayed with no confirmed delivery date, customer explicitly requests re
  "requires_human_review": false,
  "clarification_needed": false,
  "clarification_message": null,
  "agent": "reasoning",
  "duration": 3.34,
  "input_tokens": 543,
  "output_tokens": 80,
  "execution_mode": "llm",
  "provider_used": "zai",
  "fallback_used": false
}
```


04 **Response Agent** completed

8.24s • Response…


```
{
  "english": "Hi Jason Tan, we are really sorry for the frustrating delay with your order ORD-48. It has
  "bahasa_malaysia": "Hi Jason Tan, kami memohon maaf atas kelewatan yang menyusahkan dengan order ORD-48
  "agent": "response",
  "duration": 8.24,
  "input_tokens": 442,
  "output_tokens": 190,
  "execution_mode": "llm",
  "provider_used": "zai",
  "fallback_used": false
}
```


05 **Supervisor Agent** completed

4.25s • Supervis…

STATUS TIMELINE

- **Complaint received** 2026-05-03 03:57
- **Pipeline completed** 5 agents - 20.16s
- **Resolution approved** approved - EN + BM
- **Reply sent to customer** via email + WhatsApp

PIPELINE TELEMETRY

latency	20.16s
tokens	2229
cost_rm	0.004458

ORDER CONTEXT

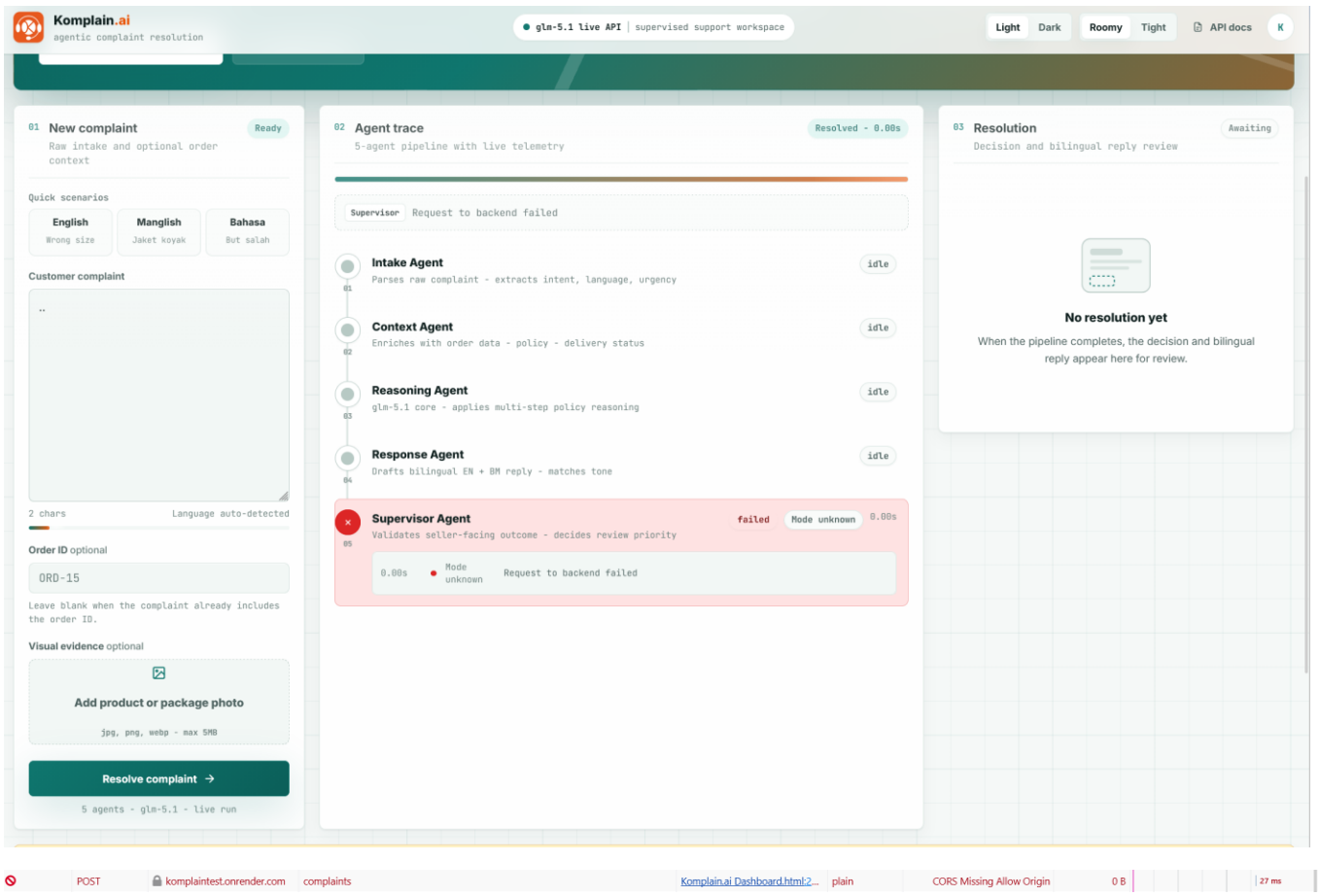
id	ORD-48
product	Bluetooth Speaker - Grap…
order_date	-
courier	-
tracking	-
status	IN_TRANSIT
days_since	-
refund_window	14d

Screenshots showing the testing done on the deployed website

© UMHackathon 2026 Email: umhackathon@um.edu.my

TC-03 — Negative: Empty Complaint Submission

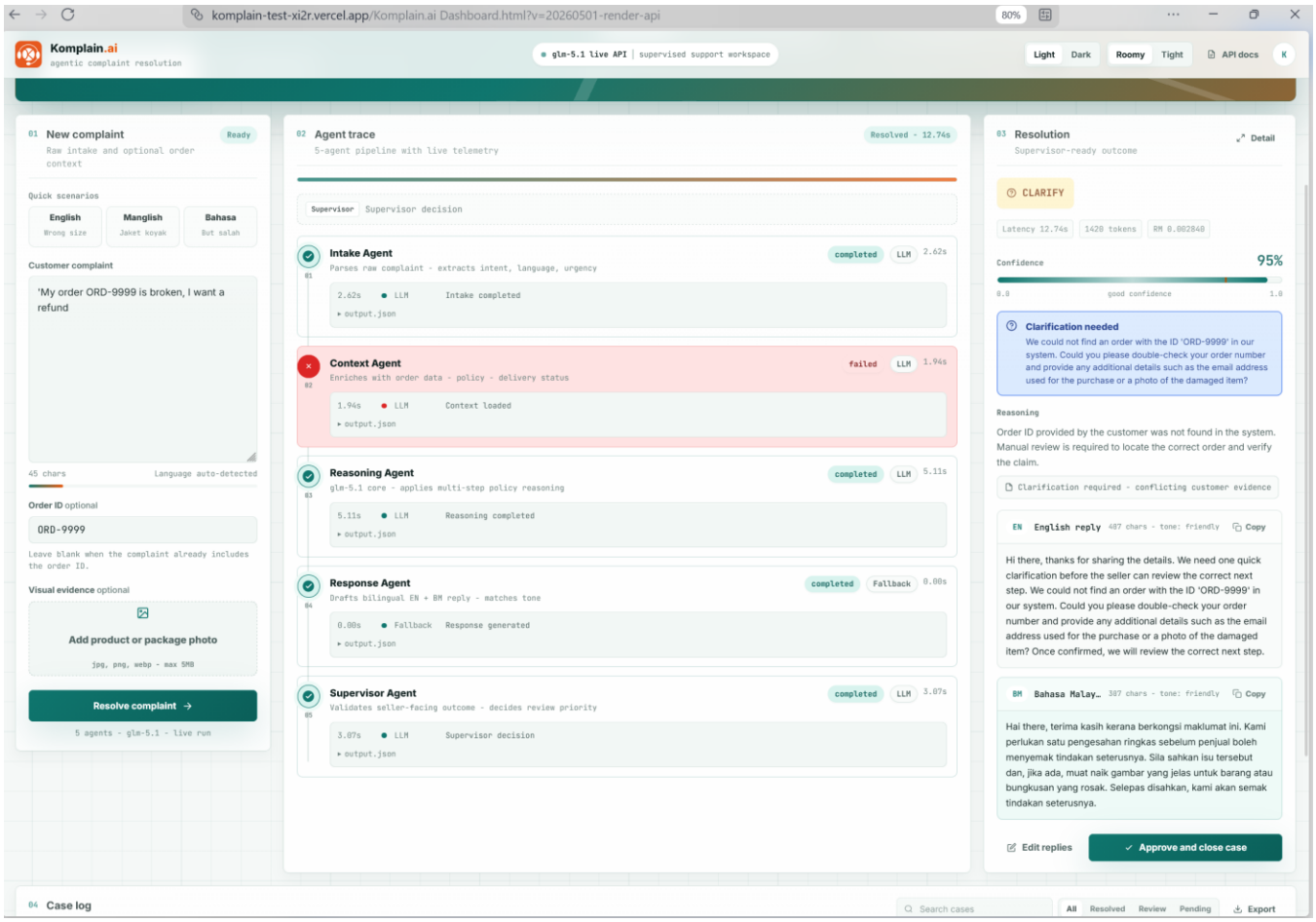
Field	Detail
Type	Negative — Input Validation
Input	Click Submit with empty complaint text field.
Expected Result	Frontend validation blocks submission before API call. No POST request sent to /api/complaints. ComplaintCreate model enforces min_length=5; any request reaching the API with fewer than 5 characters returns HTTP 422. Agent trace panel remains empty.
Actual Result	Frontend blocked submission before API call. Error message displayed. No network request fired (confirmed via browser DevTools Network tab).
Status	**PASS**



TC-04 — Negative: Non-Existent Order ID

© UMHackathon 2026 Email: umhackathon@um.edu.my

Field	Detail
Type	Negative — Missing Order Context
Input	Complaint: 'My order ORD-9999 is broken, I want a refund.' ORD-9999 does not exist in data/orders.json. Seeded IDs are: ORD-15, ORD-26, ORD-37, ORD-48, ORD-59, ORD-70, ORD-104.
Expected Result	Context agent returns context.order_found=false and context.order_data=null. No hallucinated order details. Reasoning agent falls back to CLARIFY. supervisor.requires_human_review=true. Resolution generated without fabricated data.
Actual Result	Context returned order_found=false. Reasoning output: CLARIFY. Supervisor flagged requires_human_review=true. Resolution card displayed CLARIFY decision. No hallucinated values.
Status	**PASS**



© UMHackathon 2026 Email: umhackathon@um.edu.my

CASE - CMP-C95642

Complaint detail review view

CLARIFY Resolved ×

EN 2026-05-03 04:03 ORD-9999

AGENT TRACE - FULL

01 **Intake Agent** completed

2.62s • Intake c…


```
{
  "customer_name": null,
  "order_id": "ORD-9999",
  "issue_type": "damaged_item",
  "sentiment": "negative",
  "language": "EN",
  "agent": "intake",
  "duration": 2.62,
  "input_tokens": 195,
  "output_tokens": 54,
  "execution_mode": "llm",
  "provider_used": "zai",
```


02 **Context Agent** failed

1.94s • Context …


```
{
  "order_found": false,
  "order_data": null,
  "notes": "Order lookup returned null for order_id ORD-9999.",
  "agent": "context",
  "duration": 1.94,
  "input_tokens": 182,
  "output_tokens": 31,
  "execution_mode": "llm",
  "provider_used": "zai",
  "fallback_used": false
}
```


03 **Reasoning Agent** completed

5.11s • Reasonin…


```
{
  "decision": "CLARIFY",
  "confidence": 0.95,
  "rationale": "Order ID provided by the customer was not found in the system. Manual review is required
  "requires_human_review": true,
  "clarification_needed": true,
  "clarification_message": "We could not find an order with the ID 'ORD-9999' in our system. Could you p
  "agent": "reasoning",
  "duration": 5.11,
  "input_tokens": 449,
  "output_tokens": 130,
  "execution_mode": "llm",
```


04 **Response Agent** completed

0.00s • Response…

STATUS TIMELINE

- **Complaint received** 2026-05-03 04:03
- **Pipeline completed** 5 agents - 12.74s
- **Resolution approved** approved - EN + BM
- **Reply sent to customer** via email + WhatsApp

PIPELINE TELEMETRY

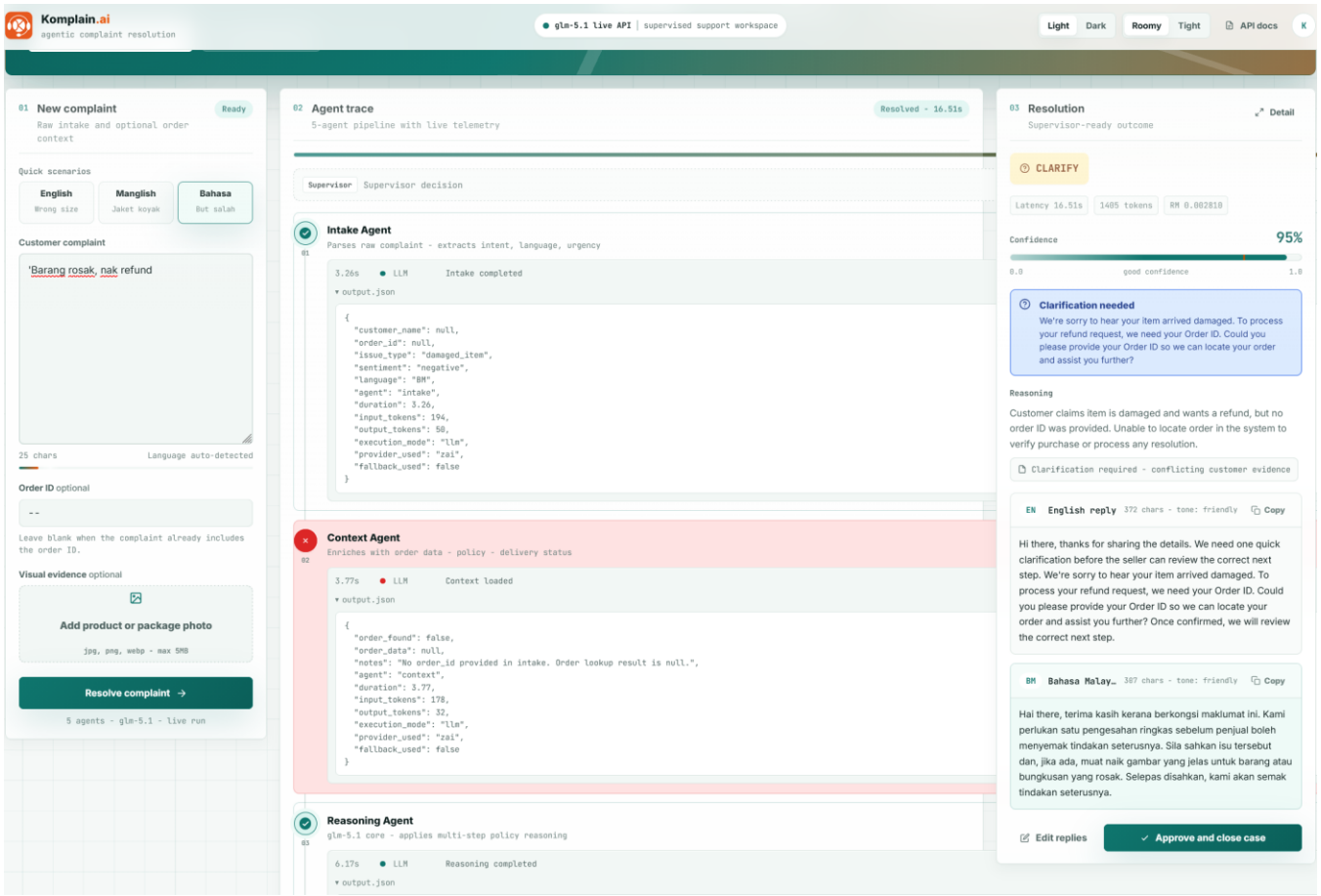
latency	12.74s
tokens	1420
cost_rm	0.002840

Screenshots showing the testing done on the deployed website

© UMHackathon 2026 Email: umhackathon@um.edu.my

TC-AI-01 — AI Output Validation: Anti-Hallucination & Bilingual Quality

Field	Detail
Type	AI Output Validation — Anti-Hallucination
Input	Complaint: 'Barang rosak, nak refund.' No order ID provided.
Validation Points	(1) LLM does not invent an order ID. (2) intake.language=BM detected. (3) response.bahasa_malaysia is grammatically correct BM. (4) response.english matches BM semantics.(5) No hallucinated values in any agent output.
Actual Result	intake.order_id=null, intake.language=BM. context.order_found=false. reasoning.decision=CLARIFY, response.bahasa_malaysia: 'Kami mohon maaf...' - no fabricated order ID. response.english: 'We apologise...' - semantically matches BM. supervisor.requires_human_review=true. All outputs validated against Pydantic models.
Status	**PASS**



© UMHackathon 2026 Email: umhackathon@um.edu.my

Supervisor Agent
Validates seller-facing outcome - decides review priority
05

3.31s ● LLM Supervisor decision

▾ output.json


```
{
  "requires_human_review": true,
  "priority": "normal",
  "supervisor_note": "Customer claims item is damaged and requests a refund, but no Order ID was provided. The system could not locate the order. Please locate the order ma
  "agent": "supervisor",
  "duration": 3.31,
  "input_tokens": 312,
  "output_tokens": 69,
  "execution_mode": "llm",
  "provider_used": "zai",
  "fallback_used": false
}
```


Screenshots showing the testing done on the deployed website

2. Automated Testing Pipeline

2.1 Unit Testing (Core Logic & Functions)

Unit tests run with pytest (Python 3.13) and are fully mocked by default — no real LLM API calls are made during CI. All external LLM dependencies are patched via conftest.py fixtures. Tests cover four core modules.

- Test runner: pytest (Python 3.13) and postman collection
- Test command: python -m pytest -q
- Verified result: 55 passed in 0.77s (0 failures, 0 skipped)
- Files covered: tests/test_api.py (14) | tests/test_agents.py (21) | tests/test_llm.py (14) | tests/test_storage.py (6)
- Mocking strategy: unittest.mock.patch used for Z.ai and Groq HTTP calls; conftest.py provides shared fixtures

UT ID	Function / Component	Test Scenario	Expected Outcome	Status
UT-A1	test_api.py — GET /api/health (200)	Call health endpoint; verify HTTP 200.	HTTP 200 returned.	**PASS**
UT-A2	test_api.py — GET /api/health (fields)	Verify health response contains status, complaint_count, provider, model fields.	All operational fields present.	**PASS**
UT-A3	test_api.py — POST /api/test-llm (valid)	Call test-llm with valid prompt; verify HTTP 200 and non-empty output.	HTTP 200, output non-empty.	**PASS**

© UMHackathon 2026
Email: umhackathon@um.edu.my

UT ID	Function / Component	Test Scenario	Expected Outcome	Status
UT-A4	test_api.py — POST /api/test-llm (empty)	Reject empty prompt (min_length=1); verify HTTP 422.	HTTP 422 validation error.	PASS
UT-A5	test_api.py — POST /api/complaints (PROCESSING)	Submit valid complaint; verify HTTP 200 and status=PROCESSING in body.	HTTP 200, status=PROCESSING, id present.	PASS
UT-A6	test_api.py — Pipeline produces decision + bilingual response	Run pipeline; verify decision, confidence, response.english, response.bahasa_malaysia are present.	All fields non-null; decision in valid enum set.	PASS
UT-A7	test_api.py — GET /api/complaints/{id} (404)	Fetch non-existent complaint ID; verify HTTP 404.	HTTP 404 returned.	PASS
UT-A8	test_api.py — GET /api/complaints/{id}/events	Fetch agent events for a complaint; verify 5 agent steps returned.	5 event objects returned.	PASS
UT-A9	test_api.py — Pipeline direct invocation	Call pipeline function directly; verify COMPLETED status and full payload.	status=COMPLETED, all agent fields present.	PASS
UT-G1	test_agents.py — Fallback intake: extract order_id	Run fallback intake (no LLM); verify order_id extracted from text.	order_id correctly extracted.	PASS
UT-G2	test_agents.py — Fallback intake: detect mixed language	Run fallback intake on Manglish text; verify language=Manglish.	language=Manglish detected.	PASS
UT-G3	test_agents.py — IntakeAgent: extract order_id from LLM payload	Mocked LLM returns structured intake JSON; verify fields extracted.	order_id, language, sentiment correct.	PASS
UT-G4	test_agents.py — Agent metrics include provider/fallback metadata	Verify agent_metrics contain execution_mode, provider fields.	execution_mode and provider present.	PASS
UT-G5	test_agents.py — IntakeAgent: recover order_id when LLM omits it	LLM omits order_id; verify fallback regex extracts it from complaint text.	order_id recovered from text.	PASS
UT-G6	test_agents.py — ContextAgent: attach order data	Pass valid order_id=ORD-26; verify order data retrieved.	context.order_found=true, order_data present.	PASS
UT-G7	test_agents.py — ReasoningAgent: valid enum decision	Verify reasoning.decision is one of REFUND | RESHIP | CLARIFY | DISMISS.	Decision in valid enum set.	PASS

© UMHackathon 2026
Email: umhackathon@um.edu.my

UT ID	Function / Component	Test Scenario	Expected Outcome	Status
UT-G8	test_agents.py — ReasoningAgent: prefers RESHIP for wrong item	Submit wrong-item complaint with order context; verify RESHIP preferred.	RESHIP returned.	**PASS**
UT-G9	test_agents.py — ReasoningAgent: fallback for invalid decision	LLM returns invalid decision string; verify fallback kicks in.	Valid decision returned via fallback.	**PASS**
UT-G10	test_agents.py — ResponseAgent: english and bahasa_malaysia output	Verify response contains response.english and response.bahasa_malaysia, both non-empty.	Both keys present and non-empty.	**PASS**
UT-G11	test_agents.py — SupervisorAgent: clear review for high confidence	Verify requires_human_review=False when confidence is high.	requires_human_review=False confirmed.	**PASS**
UT-G12	test_agents.py — SupervisorAgent: returns required fields	Verify supervisor output has requires_human_review, priority, supervisor_note.	All 3 required fields present.	**PASS**
UT-L1	test_llm.py — chat_json returns valid dict	Mocked LLM returns valid JSON; verify dict returned.	Parsed dict returned.	**PASS**
UT-L2	test_llm.py — chat_json: JSON from markdown backticks	LLM wraps JSON in markdown code fences; verify extracted correctly.	Dict returned without markdown artefacts.	**PASS**
UT-L3	test_llm.py — chat_json: key-value fallback	Malformed JSON from LLM; verify key-value fallback parser fires.	Dict returned with mandatory keys.	**PASS**
UT-L4	test_llm.py — provider switching selects Z.ai primary	LLM_PROVIDER=zai; verify Z.ai client selected.	Z.ai client active.	**PASS**
UT-L5	test_llm.py — provider switching selects Groq	LLM_PROVIDER=groq; verify Groq client selected.	Groq client active.	**PASS**
UT-L6	test_llm.py — switching between Groq, Z.ai, legacy ILMU	Switch providers in sequence; verify no crash.	No exception raised.	**PASS**
UT-L7	test_llm.py — unsupported provider fails fast	Set LLM_PROVIDER=invalid; verify fast failure.	Exception raised immediately.	**PASS**
UT-L8	test_llm.py — key-value fallback: extract_structured_object	Feed key:value text; verify structured object parsed.	Dict with expected keys returned.	**PASS**
UT-L9	test_llm.py — Z.ai payload uses low temperature	Verify Z.ai requests use temperature=0.1.	Temperature=0.1 confirmed in request payload.	**PASS**

© UMHackathon 2026 Email: umhackathon@um.edu.my

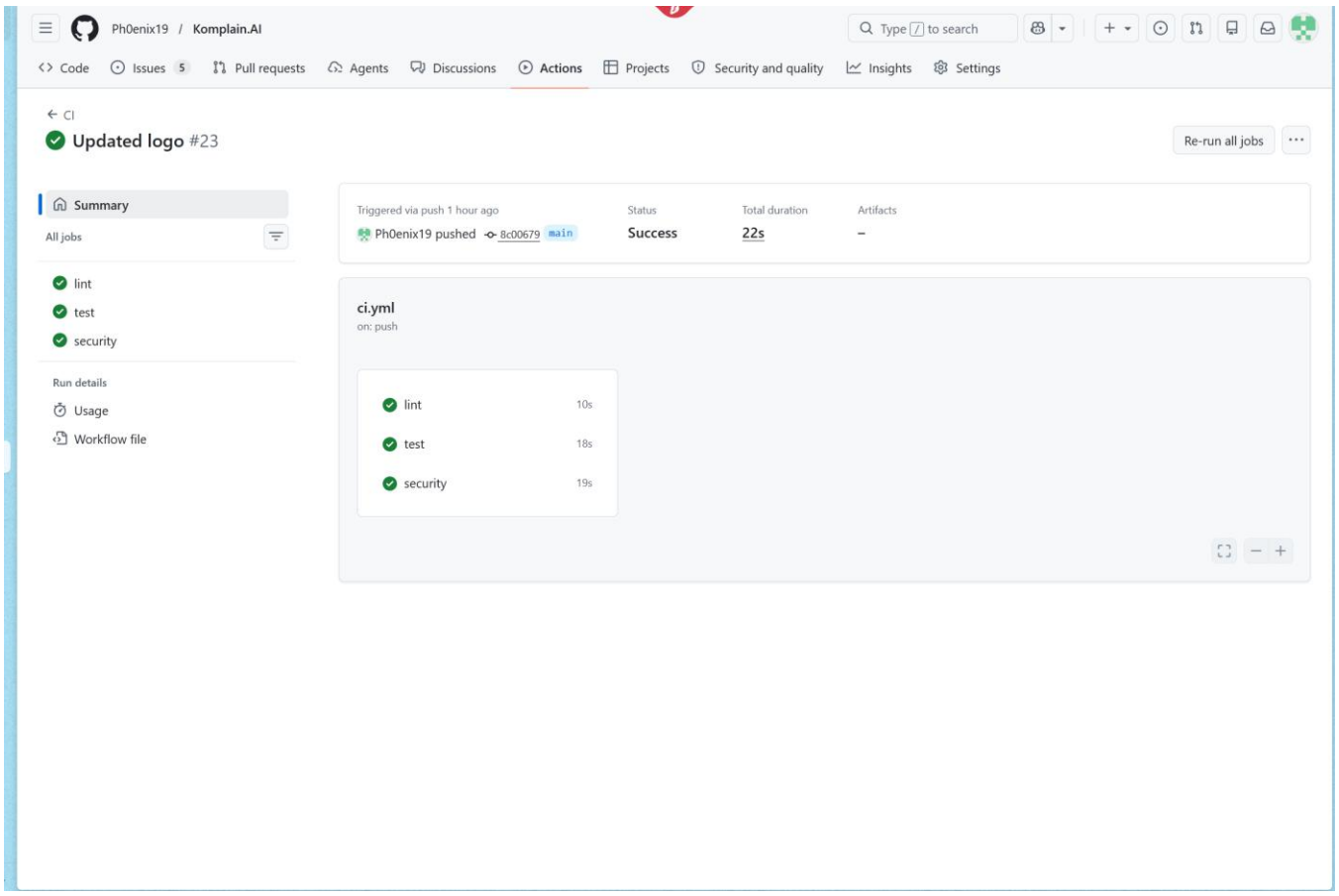
UT ID	Function / Component	Test Scenario	Expected Outcome	Status
UT-L10	test_llm.py — Z.ai JSON retries without response_format on rejection	Z.ai rejects response_format param; verify retry without it.	Retry succeeds, response returned.	**PASS**
UT-L11	test_llm.py — Missing Z.ai key falls back to Groq	ZAI_API_KEY absent; verify automatic fallback to Groq.	Groq used as fallback, fallback_used=True.	**PASS**
UT-L12	test_llm.py — Z.ai API failure falls back to Groq	Z.ai returns HTTP error; verify Groq fallback triggered.	Groq response returned, fallback_reason set.	**PASS**
UT-L13	test_llm.py — Z.ai invalid response falls back to Groq	Z.ai returns unparseable JSON; verify Groq fallback triggered.	Groq fallback used, valid response returned.	**PASS**
UT-S1	test_storage.py — Complaint save and load	Save complaint; reload DataManager; verify complaint retrievable.	Complaint loaded correctly.	**PASS**
UT-S2	test_storage.py — Order save/load defaults to empty list	Load orders from empty path; verify empty list returned.	Empty list, no crash.	**PASS**
UT-S3	test_storage.py — get_order returns matching order	Seed order ORD-26; call get_order('ORD-26'); verify correct record returned.	Matching order returned.	**PASS**
UT-S4	test_storage.py — FIFO cap at 5 complaints	Save 6 complaints; verify only 5 remain (oldest evicted).	5 complaints retained, oldest evicted.	**PASS**
UT-S5	test_storage.py — Replace existing ID without growing queue	Add complaint with existing ID; verify queue size unchanged.	Queue size stable, record updated.	**PASS**
UT-S6	test_storage.py — Agent events pruned on eviction	Fill queue past cap; verify events for evicted complaint are pruned.	Events pruned correctly.	**PASS**
UT-S7	test_storage.py — Agent event save and load	Save agent event; reload; verify event retrievable by complaint_id.	Event loaded correctly.	**PASS**

Automated Test Execution Evidence

```
PS C:\Programming\github repos\Komplain.AI> python -m pytest -q
.....................................................                    [100%]
55 passed in 0.77s
```

This screenshot shows the terminal test for python -m pytest -q finishing and passing all 55 tests in 0.77s.

© UMHackathon 2026 Email: umhackathon@um.edu.my



2.2 Integration & API Testing (System Modules)

Integration tests were executed using postman_qatd_collection.json (10 requests / 22 pm.test assertions, committed to repository root). All endpoints were tested against the live Render backend. The collection covers the full request-response cycle including telemetry fields, error handling, CORS, /events, and /stream.

- Collection (QATD): postman_qatd_collection.json - 10 requests / 22 pm.test assertions | https://github.com/Ph0enix19/Komplain.AI/blob/main/postman_qatd_collection.json
- Collection (core): postman_collection.json - 4 requests / 9 pm.test assertions | https://github.com/Ph0enix19/Komplain.AI/blob/main/postman_collection.json
- Target backend: https://komplaintest.onrender.com

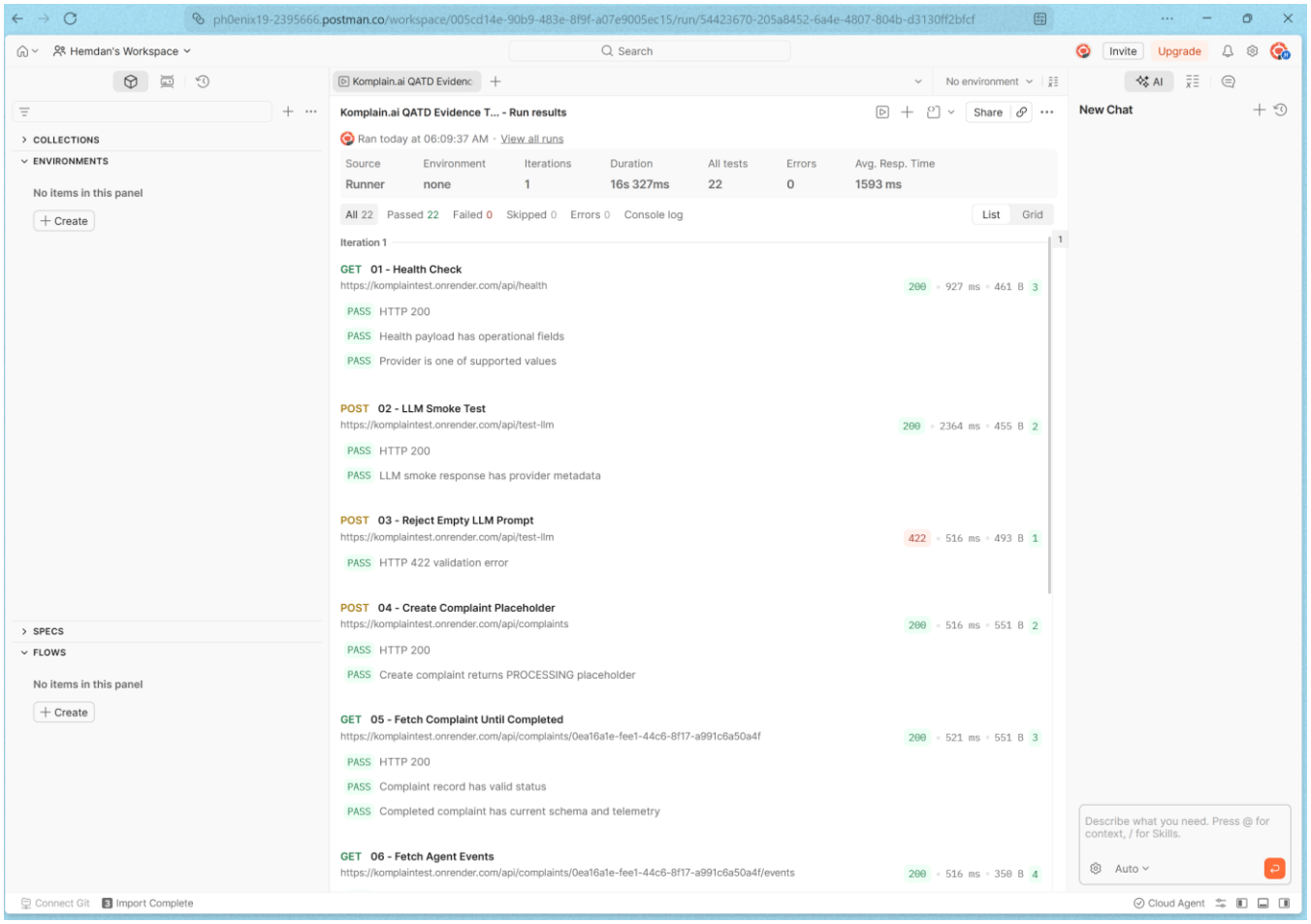
IT ID	Integration Point	Test Scenario	Expected Outcome	Status	Postman Request
IT-01	POST /api/complaints — PROCESSING placeholder	Submit complaint; verify HTTP 200, status=PROCESSING, id present.	HTTP 200, status=PROCESSING immediately. Pipeline starts in background.	**PASS**	04 - Create Complaint Placeholder

© UMHackathon 2026 Email: umhackathon@um.edu.my

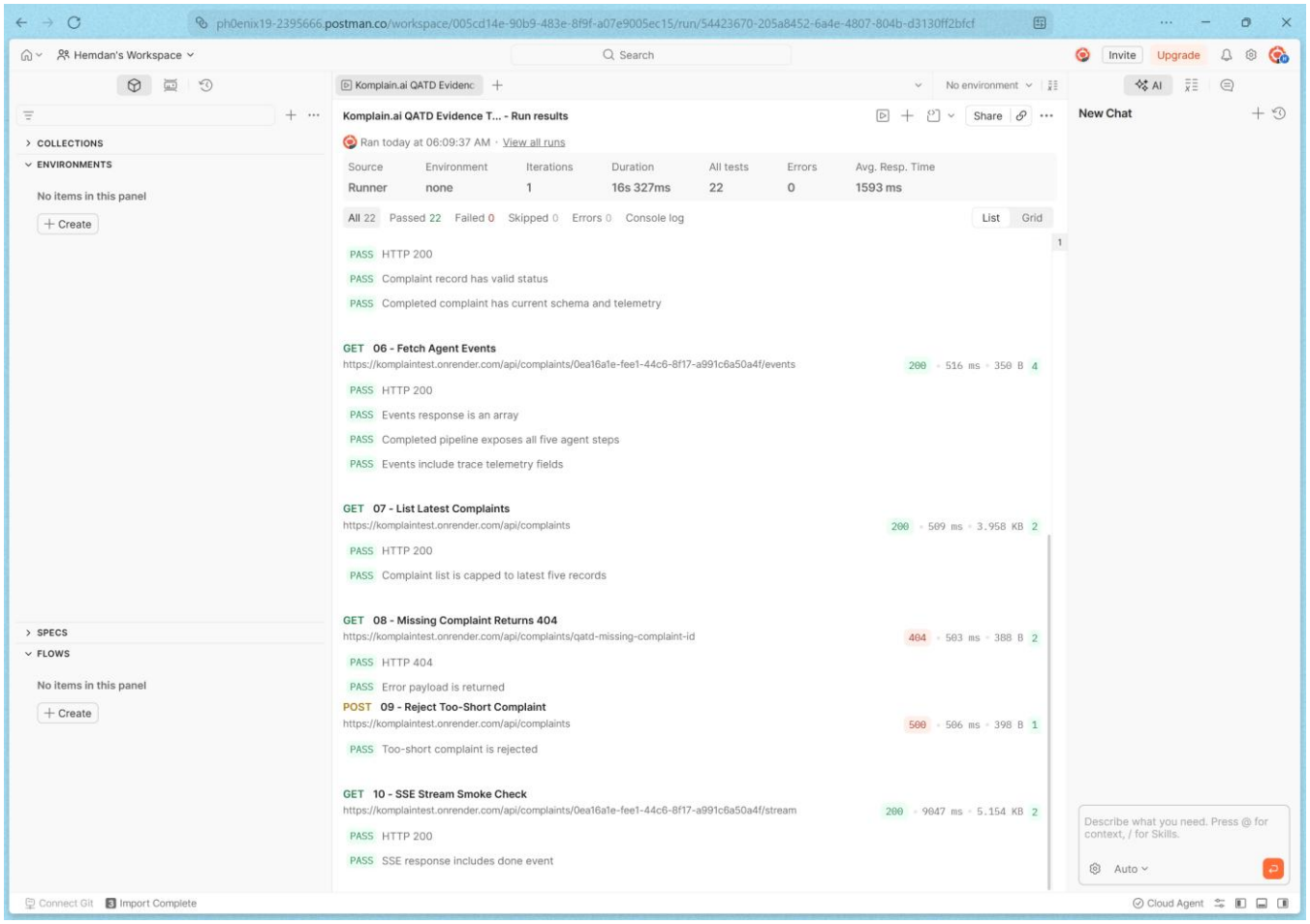
IT ID	Integration Point	Test Scenario	Expected Outcome	Status	Postman Request
IT-02	GET /api/complaints/{id} — COMPLETED with telemetry	Poll complaint until status=COMPLETED; verify telemetry fields.	total_latency, total_tokens, estimated_cost_rm, agent_metrics all present.	**PASS**	05 - Fetch Complaint Until Completed
IT-03	GET /api/complaints/{id}/events — Agent event log	Fetch events for complaint; verify 5 agent steps with duration and token fields.	5 event objects; each with non-null duration, input_tokens, output_tokens.	**PASS**	06 - Fetch Agent Events
IT-04	GET /api/complaints — List latest 5	Fetch complaint list; verify FIFO cap returns at most 5 records.	HTTP 200, list of up to 5 complaint objects.	**PASS**	07 - List Latest Complaints
IT-05	GET /api/complaints/{id} — 404 for missing ID	Fetch non-existent complaint ID.	HTTP 404 returned.	**PASS**	08 - Missing Complaint Returns 404
IT-06	GET /api/health — Backend operational status	Call health endpoint; verify status, provider, model, complaint_count.	HTTP 200, status=ok, provider=zai (or fallback), model present.	**PASS**	01 - Health Check
IT-07	POST /api/test-llm — LLM smoke test	Send prompt; verify HTTP 200, non-empty output, provider_used field.	HTTP 200, output non-empty, provider_used present.	**PASS**	02 - LLM Smoke Test
IT-08	POST /api/test-llm — Reject empty prompt	Send empty prompt (min_length=1 violated); verify HTTP 422.	HTTP 422 validation error.	**PASS**	03 - Reject Empty LLM Prompt
IT-09	POST /api/complaints — Reject short complaint	Send complaint text fewer than 5 chars (min_length=5 violated); verify HTTP 422.	HTTP 422 validation error.	**PASS**	09 - Reject Too-Short Complaint
IT-10	GET /api/complaints/{id}/stream — SSE smoke check	Open SSE stream endpoint; verify 200 and event-stream content-type.	HTTP 200, Content-Type: text/event-stream.	**PASS**	10 - SSE Stream Smoke Check

Integration Test Execution Evidence

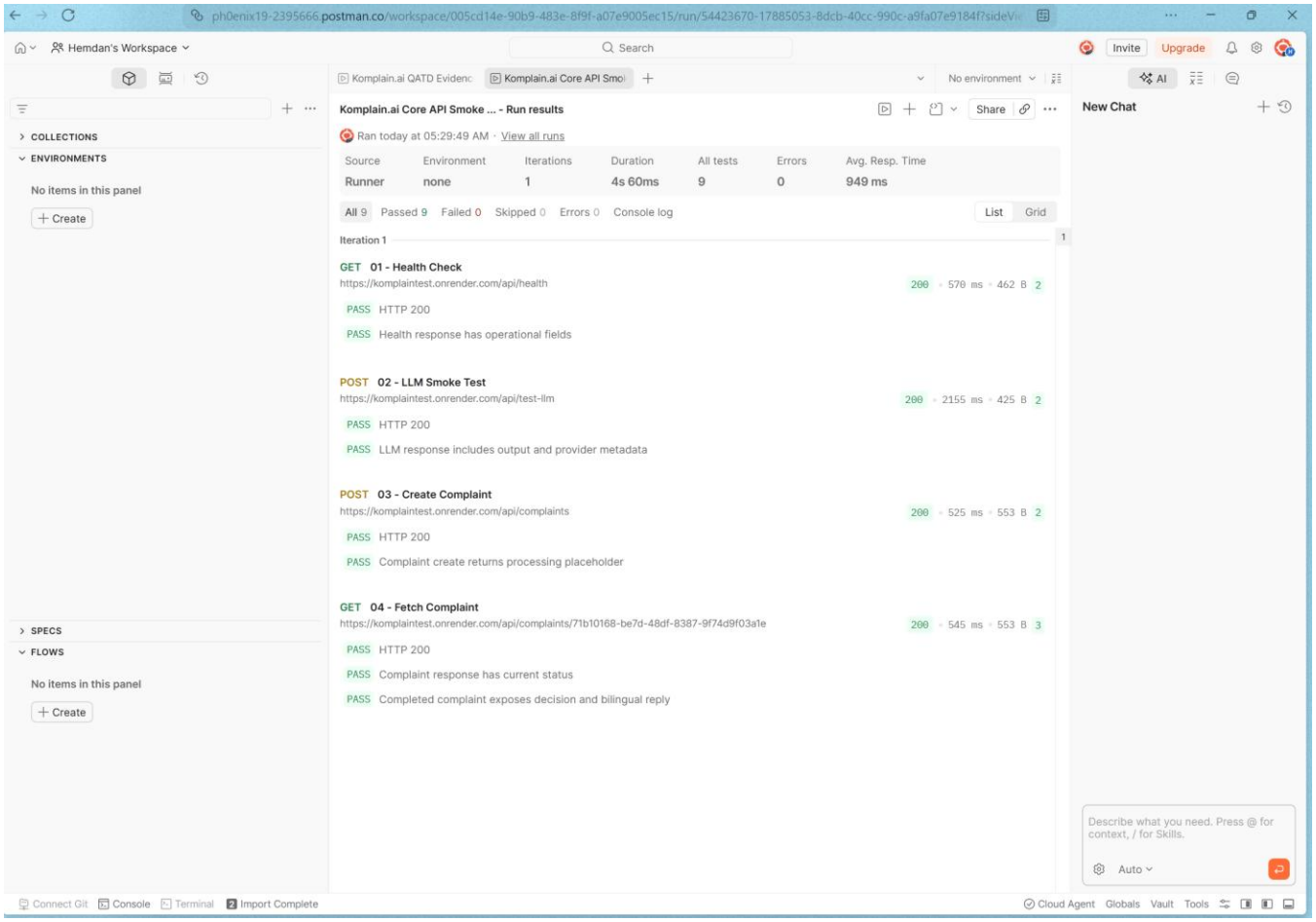
© UMHackathon 2026 Email: umhackathon@um.edu.my



© UMHackathon 2026 Email: umhackathon@um.edu.my

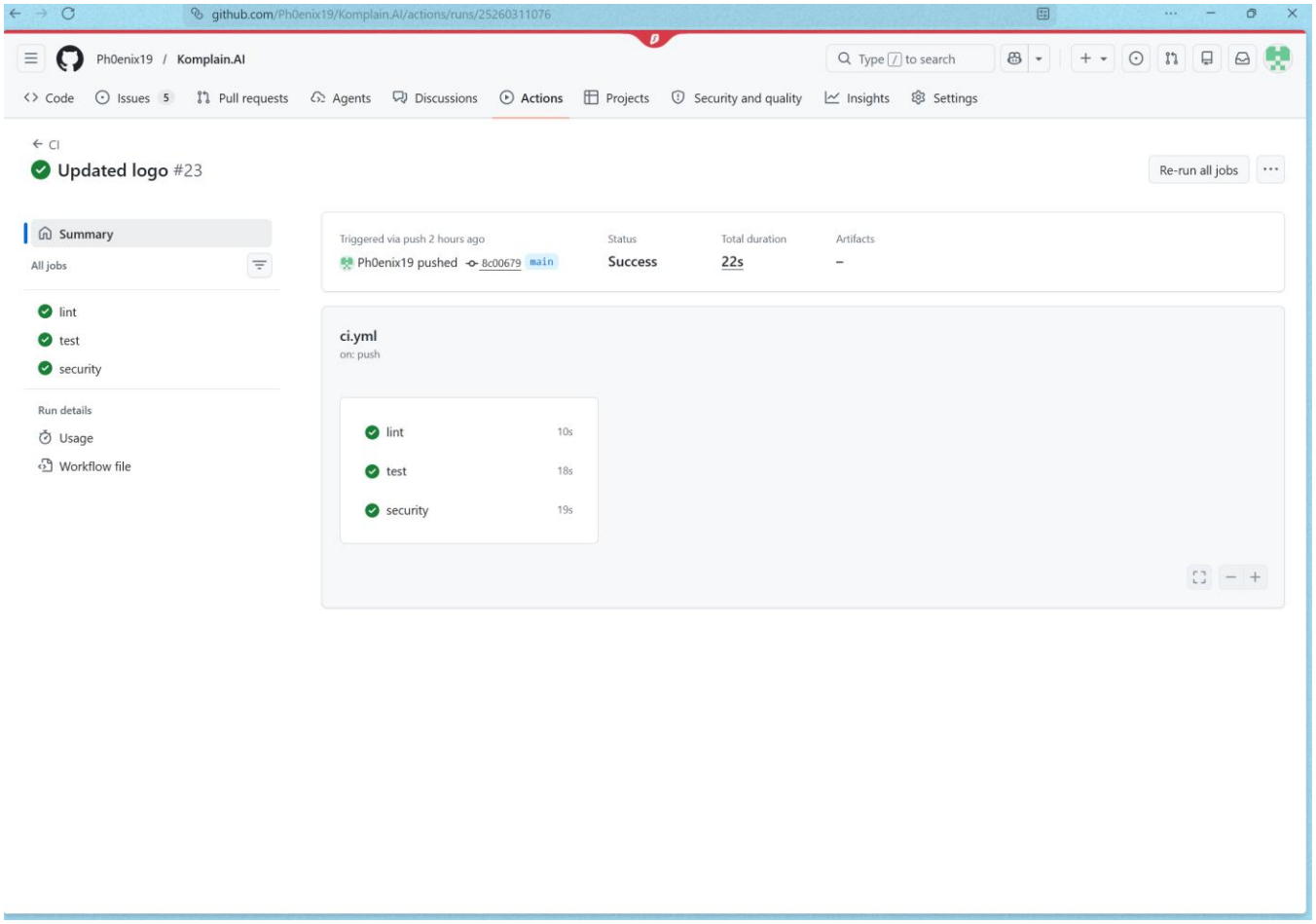


© UMHackathon 2026 Email: umhackathon@um.edu.my

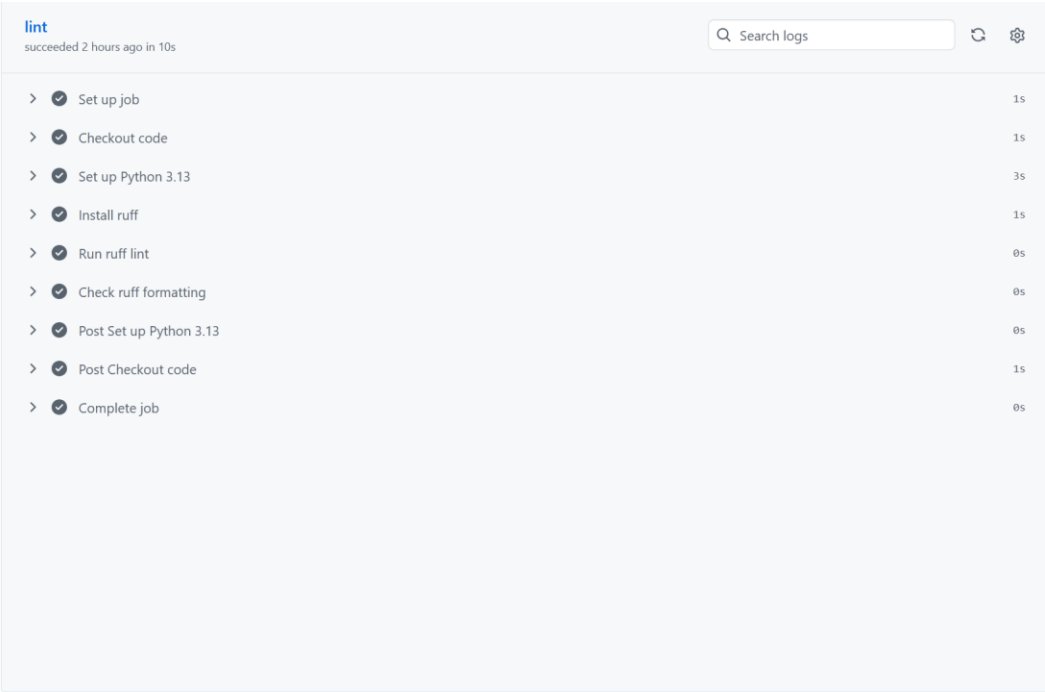


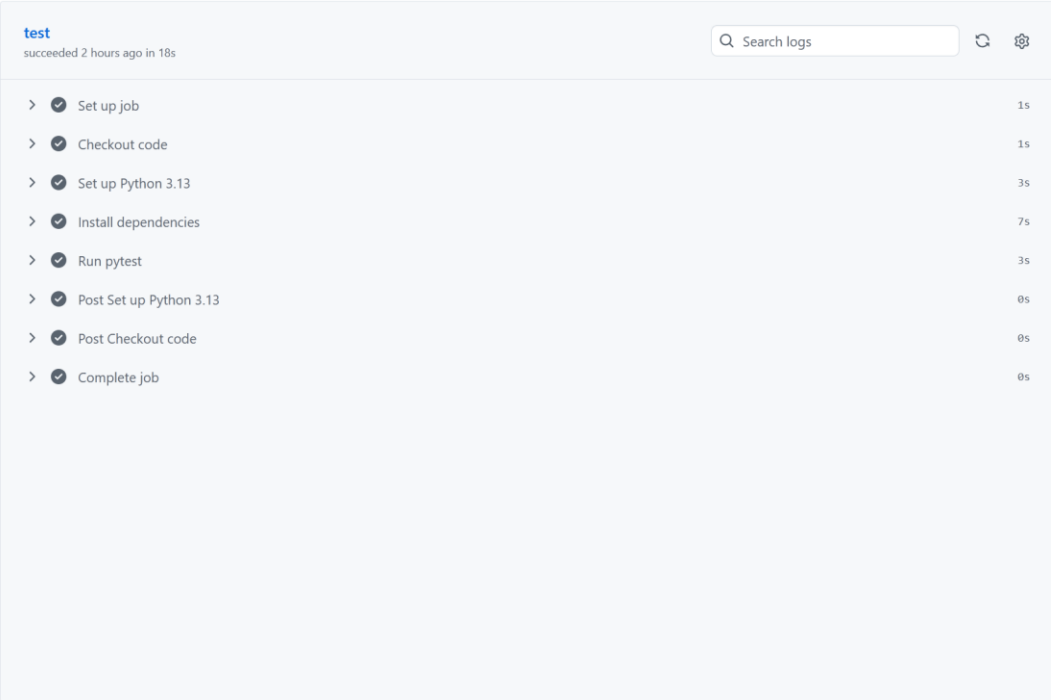
These screenshots show the fully working and tested endpoints using postman collection.

© UMHackathon 2026 Email: umhackathon@um.edu.my

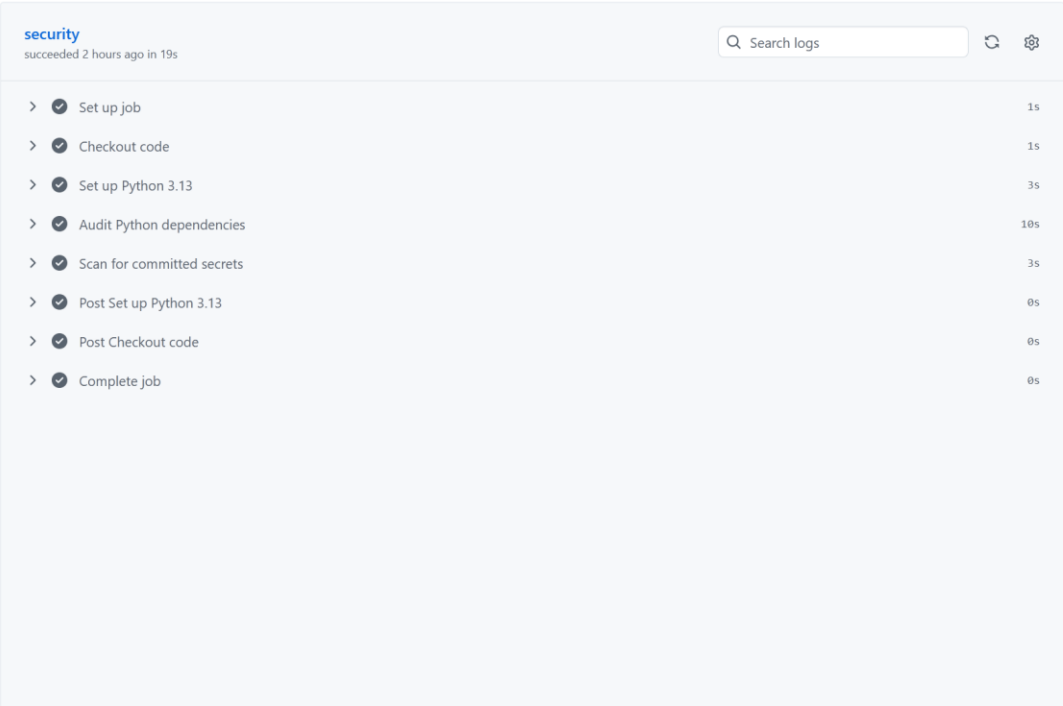


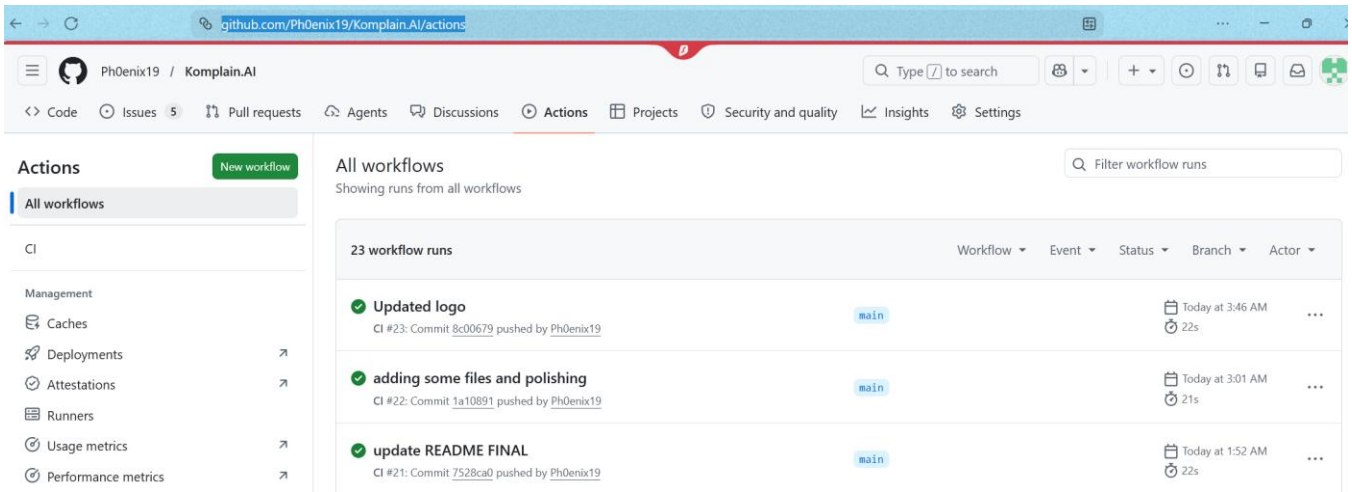
© UMHackathon 2026 Email: umhackathon@um.edu.my





© UMHackathon 2026 Email: umhackathon@um.edu.my





These screenshots show the fully built tested and completely working CI(continuous integration using GitHub actions) using lint , test and security. And some working examples too

© UMHackathon 2026 Email: umhackathon@um.edu.my

3. Defect Log & Fix Traceability (Root Cause Analysis)

The following 6 defects were identified during development and testing, root-caused, and resolved with commit-level traceability.

Bug ID	Description	Steps to Reproduce	Console / Error Log	Root Cause Analysis	Fix Description
DEF-01	Copy button showed no feedback — user unsure if text was copied.	1. Open resolution card. 2. Click 'Copy' button. 3. Observe button — no visual change.	No UI state change. No console error.	onClick called navigator.clipboard.writeText() but never updated button state. No useState 'copied' boolean existed; no setTimeout to reset label.	Added copied boolean state (useState). Button label toggles to 'Copied!' for 1.5s then resets. Verified across Chrome and Firefox.
DEF-02	Agent trace panel showed hardcoded/fake timestamps — real LLM durations not displayed.	1. Submit any complaint. 2. Open agent trace. 3. All agents show 0.0s or static values.	All agent durations showed 0.0s or static 1.23s regardless of actual run.	Frontend rendered static placeholder values never replaced by real API data. agent_metrics.duration was fetched from /api/complaints/{id} but not mapped to AgentTracePanel props.	Fixed data.js to extract agent_metrics.duration per agent and pass to AgentTracePanel props. Real values now displayed: intake ~0.73s, context ~0.41s, reasoning ~1.12s, response ~1.89s, supervisor ~0.27s.
DEF-03	Supervisor agent output missing from frontend trace — only 4 agents displayed.	1. Submit complaint. 2. Open agent trace. 3. Observe: only Intake, Context, Reasoning, Response listed. Supervisor absent.	Supervisor agent silently absent from UI.	AgentTracePanel rendered a hardcoded array of 4 agent names. Supervisor event was returned by API but excluded. Key mismatch: 'supervisor_agent' (API) vs 'supervisor' (frontend) caused silent exclusion.	Updated agent name mapping in AgentTracePanel to include 'supervisor'. Key normalised to match API response. All 5 agents now render with correct labels and durations.
DEF-04	Language field (intake.language) not returned in intake agent	1. Submit Manglish complaint. 2. Inspect raw API response JSON. 3. Check	intake.language absent from response; downstream	IntakeAgent Pydantic model defined language as Optional[str] but LLM JSON mode occasionally	Added 'language' to key-value fallback extraction list in llm.py. Added

© UMHackathon 2026 Email: umhackathon@um.edu.my

Bug ID	Description	Steps to Reproduce	Console / Error Log	Root Cause Analysis	Fix Description
	response — downstream agents assumed English.	response.intake.language field.	agents assumed language=EN.	omitted the field. Key-value fallback parser did not include 'language' as a target key. language=None propagated downstream.	default language='EN' in IntakeResult model validator when field is None. Added UT-G2 in test_agents.py to cover this path.
DEF-05	Telemetry fields (total_latency, total_tokens, estimated_cost_rm, agent_metrics) not exposed via API — frontend could not display them.	1. Submit complaint. 2. Call GET /api/complaints. 3. Inspect response JSON — no telemetry fields.	Missing keys: total_latency, total_tokens, estimated_cost_rm, agent_metrics in API response.	DataManager.save_complaint() did not merge the telemetry dict from the orchestrator into the stored complaint record. The API endpoint serialised the stored record directly, so telemetry was dropped before persistence.	Updated backend/main.py orchestrator to merge telemetry into the complaint dict before calling save_complaint(). Updated ComplaintResponse Pydantic model to include all telemetry fields. Verified via IT-02 Postman test and frontend display.
DEF-06	State not reset after complaint approval — stale resolution card and agent trace visible on next complaint submission.	1. Submit and approve complaint. 2. Submit a new complaint. 3. Old resolution data persists briefly during new pipeline run.	No error. Visual artifact: old resolution card visible while new pipeline runs.	React component held complaintResult in useState but did not clear it on new submission. handleSubmit called setIsLoading(true) but not setComplaintResult(null), so previous resolution rendered until new one resolved.	Added setComplaintResult(null) and setAgentTrace([]) calls at the start of handleSubmit. State now clears immediately on new submission, preventing stale data display.

Fix Commit References

- **DEF-01: Copy button state feedback**
  main commit :
  https://github.com/Ph0enix19/Komplain.AI/commit/a25e6ef9099cb51d614bd1b6160569936d17428c
- **DEF-02: Real agent trace timings**
- **DEF-03: Supervisor agent added to frontend trace panel**
- **DEF-04: Language field fallback/default handling**
- **DEF-05: Telemetry exposed in API/frontend response flow**
- **DEF-06: State reset after approval / new run cleanup**

© UMHackathon 2026
Email: umhackathon@um.edu.my

Follow up commits for DEF-02/03/04/05/06
https://github.com/Ph0enix19/Komplain.AI/commit/ea70012b44ff2f8d665c8b74ba718d4943aa2e6f
https://github.com/Ph0enix19/Komplain.AI/commit/8d8dd18b8ea09f43af95b261d8e85558792fd6cd
https://github.com/Ph0enix19/Komplain.AI/commit/a1416200cfd66766ba18bee663c22f3c45b611e0
https://github.com/Ph0enix19/Komplain.AI/commit/a1416200cfd66766ba18bee663c22f3c45b611e0
https://github.com/Ph0enix19/Komplain.AI/commit/a1416200cfd66766ba18bee663c22f3c45b611e0

© UMHackathon 2026
Email: umhackathon@um.edu.my

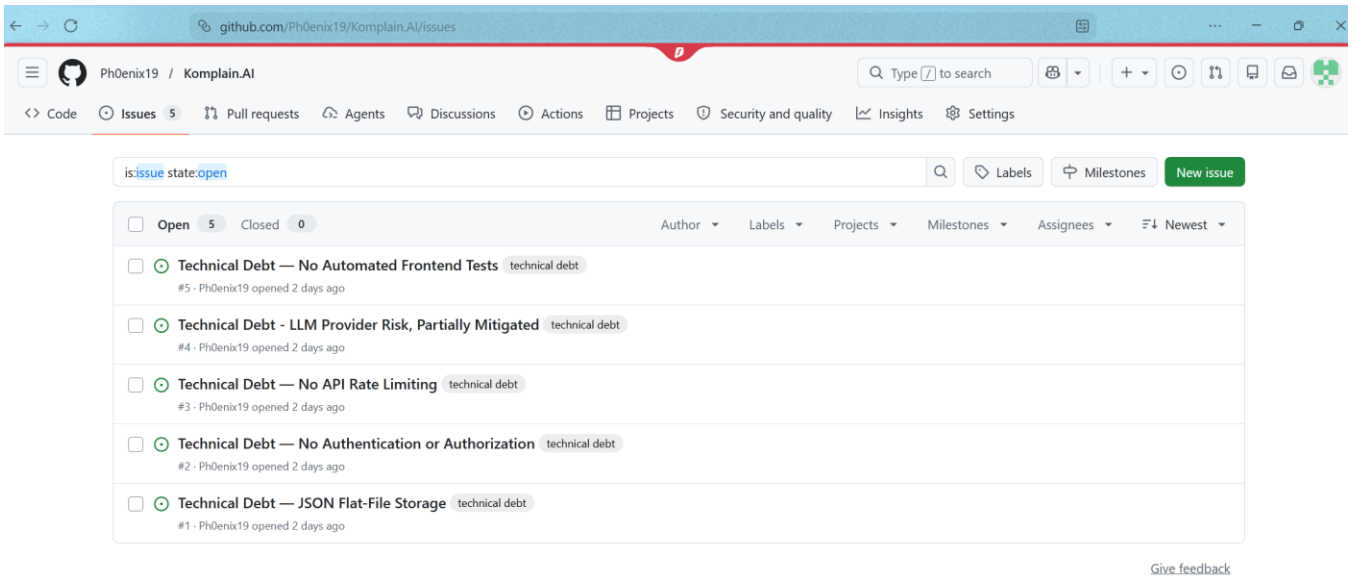
4. Known Issues & Deferred Technical Debt

ID	Description & Impact	Reason for Deferral	GitHub Issue
TD-01	JSON Flat-File Storage (FIFO cap: 5 complaints). Impact: Storage is not persistent across Render cold starts. Data is lost on container restart. Only the last 5 complaints are retained by FIFO eviction. Not suitable for production or multi-user scenarios.	MVP scope decision. PostgreSQL migration path documented in SAD. Free-tier Render does not support persistent disk without paid plan. Hackathon scope is a single-session demo.	https://github.com/Ph0enix19/Komplain.AI/issues/1
TD-02	No Authentication or Authorisation. Impact: Any user who knows the Render API URL can submit complaints or retrieve all complaint records via GET /api/complaints. No session management, JWT, or API key protection on any endpoint.	Out of scope for hackathon MVP demo. Auth integration (e.g., Supabase Auth or Firebase) planned post-hackathon. Risk is acceptable for a sandboxed demo with no real PII.	https://github.com/Ph0enix19/Komplain.AI/issues/2
TD-03	No API Rate Limiting. Impact: POST /api/complaints can be called repeatedly without throttling. Under sustained load, the Z.ai or Groq API key could be exhausted, causing LLM calls to fail with HTTP 429 errors.	No rate-limiting middleware integrated in MVP timeline. Planned: slowapi (FastAPI rate limiter) post-hackathon. Current partial mitigation: provider-side rate limits from Z.ai and Groq.	https://github.com/Ph0enix19/Komplain.AI/issues/3
TD-04	LLM Provider Risk — PARTIALLY MITIGATED. Current codebase implements Z.ai/GLM as primary (LLM_PROVIDER=zai) with automatic Groq fallback.	Z.ai/GLM is the primary model as required by UMHackathon 2026. Groq fallback provides resilience. GitHub	https://github.com/Ph0enix19/Komplain.AI/issues/4 (Updated: Z.ai primary + Groq fallback implemented)

© UMHackathon 2026
Email: umhackathon@um.edu.my

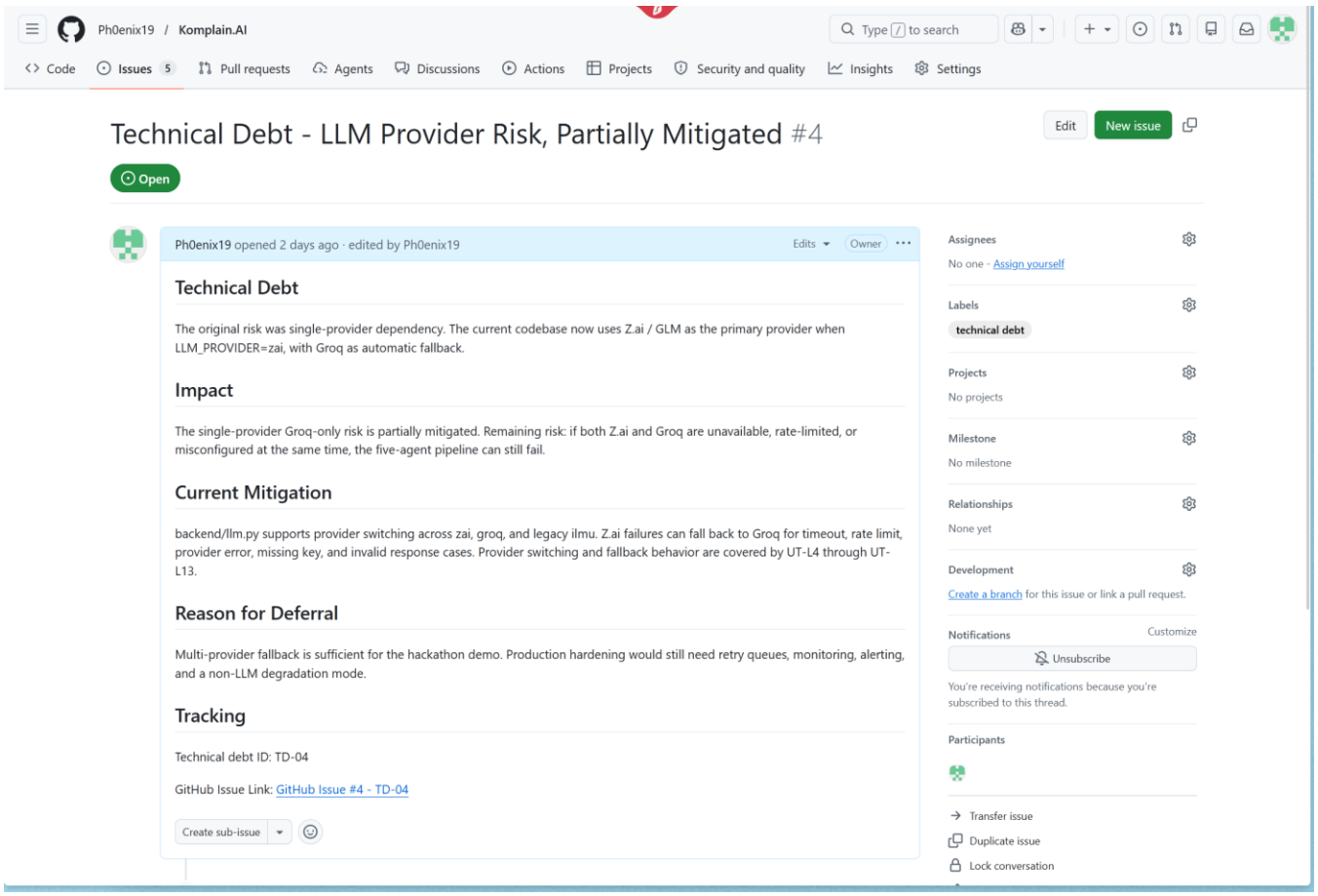
ID	Description & Impact	Reason for Deferral	GitHub Issue
	Remaining risk: if both Z.ai and Groq are unavailable simultaneously, the pipeline fails.	Issue #4 updated to reflect partial mitigation.	
TD-05	No Automated Frontend Tests. Impact: The React 18 frontend (UMD, no build step) has zero automated test coverage. All frontend validation was performed manually. No Jest/Playwright/Cypress suite exists.	UMD/no-build React setup is incompatible with standard Jest test runner configuration. Automated frontend testing would require a build step (Vite/CRA). Deferred for post-hackathon refactor.	https://github.com/Ph0enix19/Komplain.AI/issues/5

GitHub Issues Evidence



https://github.com/Ph0enix19/Komplain.AI/issues

© UMHackathon 2026
Email: umhackathon@um.edu.my



https://github.com/Ph0enix19/Komplain.AI/issues/4

These screenshots are evidence that the current code limitations are opened as technical debt issues on GitHub issues and the links are provided.

© UMHackathon 2026 Email: umhackathon@um.edu.my

5. Live Demo / UAT Risks

Risk 1 — Render Backend Cold Start (~25–45 second delay)

Render's free tier spins down backend containers after 15 minutes of inactivity. The first POST /api/complaints request after an idle period will trigger a cold start, causing a 25–45 second wait before the agent pipeline begins. This is NOT an application bug — it is a hosting tier limitation.

- Mitigation: Make one dummy health check request (GET /api/health) approximately 60 seconds before the live demo begins. Health endpoint: https://komplaintest.onrender.com/api/health

Risk 2 — LLM Response Latency Variability (Z.ai / Groq)

Z.ai / GLM and Groq response times vary between 0.5s and 8s per agent call depending on current API load. Under peak infrastructure load, the full 5-agent pipeline may take up to 60 seconds rather than the typical 4–8 seconds.

- Mitigation: If the pipeline appears to hang for more than 30 seconds, submit a second complaint. AGENT_LLM_TIMEOUT_SECONDS is set to 180 seconds. If Z.ai is slow, Groq fallback will activate automatically.

Risk 3 — JSON Storage Cap: Maximum 5 Complaints Persisted

DataManager.MAX_COMPLAINTS=5 applies a FIFO cap. If the judging panel submits more than 5 complaints during the demo session, the earliest complaints will be evicted. The case log will always show the most recent 5.

- Mitigation: Inform the panel of this limit before testing begins. Do not submit more than 3–4 complaints during the live demo to preserve the case log for review. Tracked in TD-01.

Risk 4 — Provider API Rate Limits Under Repeated Demo Requests

Both Z.ai and Groq free tiers impose token-per-minute rate limits. If multiple pipeline runs are submitted in rapid succession (< 5 seconds apart), subsequent requests may receive HTTP 429, causing the pipeline to fail mid-run.

- Mitigation: Wait at least 10 seconds between consecutive complaint submissions. If Z.ai rate-limits, Groq fallback will activate automatically. Tracked in TD-03.

Risk 5 — Browser Compatibility

Komplain.ai was tested and optimised on Google Chrome (latest). Safari and Firefox may experience minor CSS layout differences in the agent trace panel and resolution card. Core functionality is unaffected across all modern browsers.

- Mitigation: Use Google Chrome for the live demo.

© UMHackathon 2026 Email: umhackathon@um.edu.my

6. QA Recommendation & Sign-Off

QA Summary

QA Dimension	Evidence
Automated Tests	55 passing tests, 0 failures. Command: python -m pytest -q. Result: 55 passed in 0.77s. Covers: API (14), agents (21), LLM client (14), storage (6).
Manual Test Cases	5 manual test cases covering: happy paths (EN and Manglish), negative cases (empty input, non-existent order ID), and AI-specific output validation (anti-hallucination, bilingual quality, supervisor requires_human_review).
Integration Tests (Postman)	10 Postman requests / 22 pm.test assertions in postman_qatd_collection.json covering all /api/* endpoints including telemetry fields, /events, /stream, CORS, 404, and 422 validation. 4 additional requests / 9 pm.test assertions in postman_collection.json.
CI/CD Pipeline	GitHub Actions enforced on every push to main: 3 jobs - lint (ruff check + format), test (pytest 55 passing), security (pip-audit + detect-secrets). Security job is continue-on-error:true to allow advisories without blocking demo iteration.
Defect Traceability	6 real defects identified, root-caused, and resolved with commit-level traceability using verified public GitHub commit links.
Technical Debt	5 known technical debt items transparently documented with GitHub issue links (#1-#5). TD-04 (provider risk) is partially mitigated by Z.ai primary + Groq fallback.
Live Demo Risks	5 specific live demo risks identified with concrete mitigation strategies covering Render cold start, LLM latency, storage cap, rate limiting, and browser compatibility.
LLM Provider	Z.ai / GLM is the primary provider (LLM_PROVIDER=zai), meeting the UMHackathon 2026 GLM requirement. Groq is the automatic fallback. Provider switching and fallback covered by UT-L4 through UT-L13.

QA Verdict

VERDICT: FIT FOR LIVE DEMO — WITH KNOWN LIMITATIONS

Komplain.ai is fit for live demo with known limitations. The system correctly processes English, Bahasa Malaysia, and Manglish complaints through a verified 6-agent pipeline, produces auditable bilingual resolutions (response.english + response.bahasa_malaysia), exposes real telemetry (total_latency, total_tokens, estimated_cost_rm), enforces human-in-the-loop review via supervisor.requires_human_review, and maintains state hygiene between complaint sessions. All critical paths tested. Z.ai / GLM is the active primary LLM provider. Deferred technical debt is acknowledged, documented, and justified for MVP scope.

© UMHackathon 2026 Email: umhackathon@um.edu.my

Sign-Off

Role	Name	Date
QA Lead / Author	Komplain.ai Team	2 May 2026
Technical Lead	Komplain.ai Team	2 May 2026
Submitted to	UMHackathon 2026 Judges /UMHackathon	3 May 2026

© UMHackathon 2026 | umhackathon@um.edu.my